

## **Instituto Superior de Engenharia de Lisboa**

Departamento de Engenharia de Electrónica e Telecomunicações e de Computadores

Licenciatura em Engenharia Informática, Redes e Telecomunicações

### **Relatório Inicial de Projeto**

**API de comunicação para *smart tool cabinets***

|                           |                               |
|---------------------------|-------------------------------|
| <b>Unidade curricular</b> | Projeto - LEIRT               |
| <b>Aluno</b>              | Ricardo Nobre (48713)         |
| <b>Orientadores</b>       | Tiago Dias e Carlos Gonçalves |
| <b>Ano letivo</b>         | 2025/2026                     |

Lisboa, março de 2026

## 1. Enquadramento

O projeto enquadra-se no desenvolvimento de um sistema central para gestão e monitorização de *smart tool cabinets* utilizados em oficinas de manutenção de sistemas críticos. Esse sistema deverá acompanhar a utilização dos armários e das ferramentas, manter histórico das operações e ajudar a identificar situações que possam comprometer a segurança no local de trabalho.

O trabalho a desenvolver no âmbito deste projeto incide sobre a componente responsável pela comunicação entre os armários e o sistema central, incluindo o desenvolvimento da API de comunicação, da lógica de *backend*, da persistência de dados e de simuladores para interação com o sistema. Outras partes do sistema, como a interface de gestão para utilizadores, situam-se fora do âmbito deste projeto.

No cenário considerado para o projeto, cada armário pode ser utilizado por um utilizador autenticado por PIN ou por cartão NFC. Cada ferramenta possui um identificador RFID único e a respetiva utilização fica associada a uma sessão aberta nesse armário. Quando o utilizador tenta fechar o armário, o sistema deverá validar se as ferramentas esperadas se encontram novamente presentes. Caso se verifique a ausência de ferramentas, o fecho deverá ser recusado e a situação terá de ficar registada.

Esta delimitação do problema permite manter o foco na componente principal do sistema. A aplicação *web* ou móvel constitui uma possibilidade de evolução futura, mas não integra esta primeira versão funcional.

## 2. Descrição do problema

O problema principal deste projeto consiste em definir e implementar uma API que permita a comunicação entre os armários e o sistema central de forma consistente, simples de utilizar e suficientemente robusta para suportar o funcionamento esperado. Na prática, isto implica determinar que informação é enviada pelos armários, em que momentos é enviada, que validações devem existir e de que modo essa informação fica registada do lado do sistema central.

A API deverá suportar o ciclo base de utilização do sistema: identificação do armário, autenticação do utilizador, abertura de sessão, receção de eventos emitidos pelo armário, leitura

do estado das ferramentas, tentativa de fecho e encerramento da sessão com persistência do resultado. Para que esse funcionamento fique corretamente definido, torna-se necessário modelar entidades como armários, utilizadores, ferramentas, sessões e eventos, bem como as respetivas regras de negócio.

Entre os aspetos mais relevantes nesta definição destacam-se a unicidade global dos identificadores RFID, a associação entre ferramenta e armário, o conceito de sessão ativa, a distinção entre utilizadores e dispositivos e a necessidade de manter rastreabilidade e auditoria das operações realizadas.

Como não se terá acesso contínuo ao hardware real durante todo o desenvolvimento, o projeto compreende o desenvolvimento de um simulador de armário compatível com a API. Esse simulador permitirá reproduzir cenários normais e de falha, testar o *backend* de forma autónoma e demonstrar o funcionamento do sistema, mesmo sem ligação direta aos armários reais.

### 3. Objetivos do projeto

O objetivo geral do projeto é projetar, desenvolver e testar uma API central para *smart tool cabinets*, acompanhada por um simulador compatível, de modo a demonstrar o funcionamento da comunicação entre armário e sistema central e a deixar uma base técnica reutilizável para evoluções futuras.

As tarefas a realizar são as seguintes:

- Estudar o funcionamento previsto do sistema e consolidar os requisitos funcionais da componente API/simulador.
- Definir o modelo do domínio - armários, utilizadores, ferramentas, sessões, eventos e auditoria - e as respetivas regras de negócio.
- Especificar formalmente a API de comunicação, documentando *endpoints*, *payloads* JSON, códigos de resposta e erros.
- Desenvolver o *backend* central com persistência em base de dados, validação das regras principais e registo histórico das operações.
- Desenvolver um simulador de armário capaz de reproduzir cenários normais e cenários de falha, incluindo abertura, remoção e devolução de ferramentas e tentativa de fecho com ferramentas em falta.

- Criar mecanismos de teste e validação da API, cobrindo regras críticas e integração com a base de dados.
- Produzir documentação técnica suficiente para permitir a utilização futura da API.

### Tecnologias adotadas

| Componente              | Tecnologia adotada                                                                                                |
|-------------------------|-------------------------------------------------------------------------------------------------------------------|
| API e lógica de negócio | Java 21, Spring Boot 3.5.x e Maven. Utilização de Spring Web, Spring Data JPA, Spring Security e Bean Validation. |
| Persistência            | PostgreSQL e Flyway para versionamento e gestão do esquema da base de dados.                                      |
| Documentação            | springdoc-openapi para geração da especificação OpenAPI e Swagger UI.                                             |
| Simulador               | Java 21 com Picocli para a interface de linha de comandos e HttpClient para comunicação com a API.                |
| Testes                  | JUnit 5 e Testcontainers para testes unitários e de integração.                                                   |
| Execução local          | Docker e Docker Compose para arrancar a API, a base de dados e os restantes serviços de apoio.                    |

A utilização de Kubernetes será considerada apenas como extensão final do projeto. Não faz parte do MVP, nesta fase o mais importante é garantir que a API e o simulador funcionam corretamente.

## 4. Planeamento do trabalho

O planeamento proposto assenta na construção faseada do núcleo essencial do sistema, evoluindo-se depois para componentes complementares e atividades de validação. Numa primeira fase, proceder-se-á à consolidação do modelo do domínio e da especificação da API; numa segunda fase, ao desenvolvimento do *backend* e do simulador; e, por fim, à execução dos testes, à documentação e à preparação dos entregáveis.

Este planeamento teve igualmente em consideração o tempo necessário para revisão, estimando-se cerca de 15 dias para etapas intermédias e até 3 semanas para a fase final. Assim, o calendário apresentado mantém alguma margem para revisões e ajustes.

## Principais tarefas

- T1 - Análise do funcionamento do sistema e definição das entidades principais.
- T2 - Especificação da API e do modelo de dados.
- T3 - Implementação da API e da lógica de negócio principal.
- T4 - Desenvolvimento do simulador de armário.
- T5 - Testes e validação funcional.
- T6 - Documentação técnica e preparação das entregas.
- T7 - Empacotamento com Docker Compose e eventual extensão final em Kubernetes.

## Calendarização

| Tarefa                                                                   | Mar | Abr | Mai | Jun | Jul |
|--------------------------------------------------------------------------|-----|-----|-----|-----|-----|
| Análise do funcionamento do sistema e definição das entidades principais | X   |     |     |     |     |
| Especificação da API e do modelo de dados                                | X   | X   |     |     |     |
| Implementação do backend                                                 |     | X   | X   |     |     |
| Documentação OpenAPI                                                     |     | X   | X   |     |     |
| Implementação do simulador                                               |     |     | X   |     |     |
| Testes unitários e de integração                                         |     |     | X   | X   |     |
| Integração, correções e estabilização                                    |     |     |     | X   |     |
| Relatório final e preparação da apresentação                             |     |     |     | X   | X   |

## Datas principais

| Nome                           | Data       |
|--------------------------------|------------|
| Relatório inicial              | 13/03/2026 |
| Versão para revisão intermédia | 17/04/2026 |
| Relatório intermédio           | 01/05/2026 |
| Versão para revisão final      | 20/06/2026 |
| Relatório final                | 11/07/2026 |